DevOps life cycle phases and the mapping of open source tools

A DevOps toolchain is a set or combination of tools that helps in the delivery, development and management of applications throughout the system's development life cycle. At the enterprise level, the software teams need to automate the entire cycle of building, provisioning and deploying test environments, including the tools, scripts and test data to ensure rapid delivery. These teams need to collaborate via the application's architecture and monitor event-based mechanisms for seamless data flow across the tool chains.

Listed below are the different stages of software/app development in the DevOps life cycle:

- Continuous planning
- Continuous development
- Version control
- Continuous testing
- Continuous integration
- Continuous deployment
- Configuration management
- Containerisation tools
- Repositories
- Release automation
- Continuous monitoring

The following sections briefly describe the phases of the DevOps life cycle and the appropriate open source products.

Continuous planning: DevOps readiness assessment across an enterprise is done during this phase. Requirements for DevOps implementation, the development approach and how this transits into operations is also done at this stage. Plans on how to deploy DevOps that involve people, processes and tools are made. The definitions of the target stage, transformation plan and execution plan are set during this phase. Companies need to continuously plan, measure and introduce business strategy and customer feedback into the development life cycle.

Continuous development: DevOps establishes the interdependence of software development and IT operations. It helps an organisation produce software and IT services more rapidly, with frequent iterations. Code development is done in any language, but maintained by using version control tools. The most popular tools used are Git, SVN, SonarQube, Maven and Ant.

Version control: Versions are maintained in a central repository that acts like a single source of truth. It helps the developers to collaborate on the 'latest committed' code and the operations team can access the same code when it plans to make a release. Whenever there is a fault during the release, the ops team can quickly roll back the deployed code and revert to the previous stable state. Git is the leading version control system. It allows developers to collaborate with each other on a distributed version control system.

Continuous testing: This promotes organisation-wide cultural changes that promote capabilities like testing early, testing faster and automated testing. Continuous testing synchronises testing and QA with development and operation,

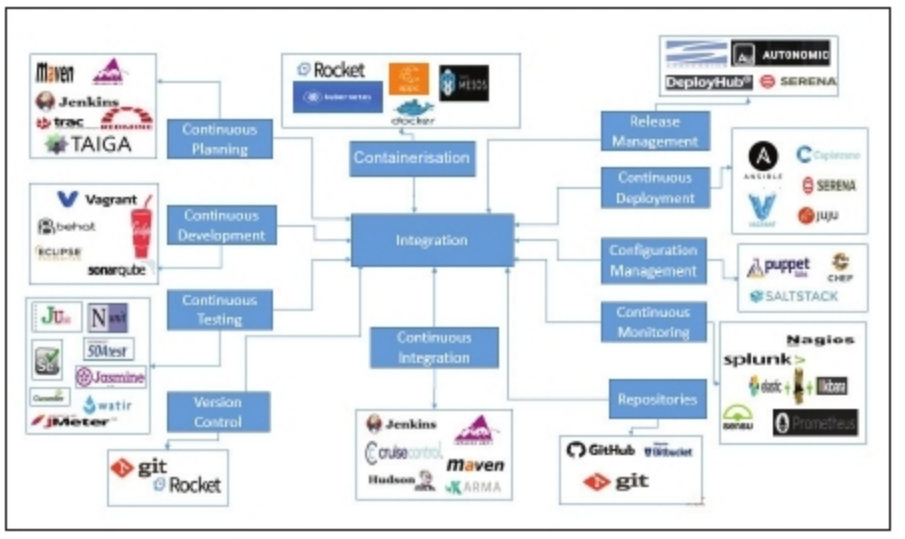

Figure 2: The DevOps life cycle and mapping of open source tools

and is optimised to achieve business and development goals. Tools like Tosca, Selenium, TestNG and JUnit are used to automate the execution of test cases.

Continuous integration (CI): This helps developers to integrate code into a shared repository several times a day. It allows teams to detect problems early and verifies each check-in. By integrating regularly, errors are detected and located easily. The most popular CI tool in the market is Jenkins. Other popular CI tools are Bamboo and Hudson.

Continuous deployment: This happens when every change goes through the pipeline and is automatically put into production, resulting in many production deployments every day with greater delivery speed and frequency for complex applications. Ansible, Kamatera and Vagrant are the most useful continuous deployment tools.

Configuration management: This helps in establishing and maintaining consistency in an application's functional requirements and performance. Configuration management tools work based on the master-slave architecture. The popular choices are Puppet, Chef, Ansible and SaltStack.

Containerisation tools: Containerisation tools help in maintaining consistency across the environments in which the application is developed, tested and deployed. It eliminates failures in the production environment by packaging and replicating the same dependencies and packages used in development, testing and the staging environment. Docker is the most used containerisation tool.

Repositories: The artefact repository is a collection of binary software artefacts and metadata stored in a defined directory structure. A repository stores two types of artefacts — releases and snapshots. Release repositories are for stable, static release artefacts, and snapshot repositories are frequently updated repositories that store binary software artefacts from projects under constant development.

GitHub is the central repository in which the code is maintained. Bitbucket and Nexus are the other repository tools.

Release automation: Deployment automation solves the problem of deploying an application by using an automated and repeatable process. DeployHub and Serena are widely used release automation tools.